

Язык программирования C++

Лекция 21: type traits, SFINAE, enable_if, concepts

Никита Лисица

2026

- Стандартный шаблон `std::iterator_traits<It>` предоставляет информацию об итераторах: категория (`random access` / `bidirectional` / `etc`), тип значения (`value_type`), и т.п.

Напоминание: `iterator_traits`

- Стандартный шаблон `std::iterator_traits<It>` предоставляет информацию об итераторах: категория (random access / bidirectional / etc), тип значения (`value_type`), и т.п.
- По умолчанию достаёт информацию из самого типа итератора `It`, но содержит специализации для указателей

Напоминание: `iterator_traits`

- Стандартный шаблон `std::iterator_traits<It>` предоставляет информацию об итераторах: категория (random access / bidirectional / etc), тип значения (`value_type`), и т.п.
- По умолчанию достаёт информацию из самого типа итератора `It`, но содержит специализации для указателей
- Такие шаблоны, содержащие некоторую информацию о типах, называются *type traits*

Напоминание: `iterator_traits`

- Стандартный шаблон `std::iterator_traits<It>` предоставляет информацию об итераторах: категория (random access / bidirectional / etc), тип значения (`value_type`), и т.п.
- По умолчанию достаёт информацию из самого типа итератора `It`, но содержит специализации для указателей
- Такие шаблоны, содержащие некоторую информацию о типах, называются *type traits*
- В заголовочном файле `<type_traits>` очень много полезных *type traits*

Type traits

```
template <typename T>
struct is_void {
    static const bool value = false;
};

template <>
struct is_void<void> {
    static const bool value = true;
};
```

Type traits

```
template <typename T>
struct is_void {
    static const bool value = false;
};

template <>
struct is_void<void> {
    static const bool value = true;
};
```

```
template <typename T>
struct is_pointer {
    static const bool value = false;
};

template <typename T>
struct is_pointer<T*> {
    static const bool value = true;
};
```

- Ещё есть `std::is_integral`, `std::is_floating_point`, `std::is_bool`, `std::is_class`, `std::is_function`, `std::is_enum`, и т.п.

- Ещё есть `std::is_integral`, `std::is_floating_point`, `std::is_bool`, `std::is_class`, `std::is_function`, `std::is_enum`, и т.п.
- Некоторые из них можно реализовать на чистом C++, но многие (напр. `std::is_enum`) реализуются через компилятор

Type traits

- Ещё есть `std::is_integral`, `std::is_floating_point`, `std::is_bool`, `std::is_class`, `std::is_function`, `std::is_enum`, и т.п.
- Некоторые из них можно реализовать на чистом C++, но многие (напр. `std::is_enum`) реализуются через компилятор
- Для всех таких traits есть вспомогательные шаблонные константы с суффиксом `_v`, чтобы не писать `::value`

```
template <typename T>
static constexpr bool is_integral_v = is_integral<T>::value;

// То же самое, что и
// is_integral<int>::value
std::is_integral_v<int>
```

- Есть вспомогательные функции на типах `std::add/remove_const`, `std::add/remove_pointer`, `std::make_signed/unsigned`, и т.п.

- Есть вспомогательные функции на типах
`std::add/remove_const`, `std::add/remove_pointer`,
`std::make_signed/unsigned`, и т.п.
- Их результат – не значение, а тип:
`std::make_signed<int>::type`

Type traits

- Есть вспомогательные функции на типах `std::add/remove_const`, `std::add/remove_pointer`, `std::make_signed/unsigned`, и т.п.
- Их результат – не значение, а тип:
`std::make_signed<int>::type`
- Для них есть вспомогательные шаблонные `using`'и с суффиксом `_t`, чтобы не писать `::type`

```
template <typename T>
using make_unsigned_t = typename make_unsigned<T>::type;

// То же самое, что и
// make_unsigned<int>::type
std::make_unsigned_t<int>
```

Type traits: использование

- Можно проверять корректность типов аргументов:

```
template <typename T>
T pow(T x, T n) {
    static_assert(std::is_integral_v<T>);
    // ...
}
```

Type traits: использование

- Можно проверять корректность типов аргументов:

```
template <typename T>
T pow(T x, T n) {
    static_assert(std::is_integral_v<T>);
    // ...
}
```

- Можно специализировать структуры данных и алгоритмы:

```
template <typename T, bool IsUnsigned = std::is_unsigned_v<T>>
class PowImpl { ... };

template <typename T>
class PowImpl<T, false> { ... };
```

Type traits: использование

- Можно проверять корректность типов аргументов:

```
template <typename T>
T pow(T x, T n) {
    static_assert(std::is_integral_v<T>);
    // ...
}
```

- Можно специализировать структуры данных и алгоритмы:

```
template <typename T, bool IsUnsigned = std::is_unsigned_v<T>>
class PowImpl { ... };

template <typename T>
class PowImpl<T, false> { ... };
```

- **NB:** и то, и другое лучше делать с помощью *концептов*

- Допустим, у нас есть шаблонная функция с набором перегрузок:

```
template <typename T>
T sum(T x, T y) {
    return x + y;
}

template <typename T>
auto sum(typename T::iterator begin, typename T::iterator end) {
    decltype(*begin) result = {};
    for (auto it = begin; it != end; ++it)
        result += *it;
    return result;
}
```

- Допустим, у нас есть шаблонная функция с набором перегрузок:

```
template <typename T>
T sum(T x, T y) {
    return x + y;
}

template <typename T>
auto sum(typename T::iterator begin, typename T::iterator end) {
    decltype(*begin) result = {};
    for (auto it = begin; it != end; ++it)
        result += *it;
    return result;
}
```

- Что произойдёт, если вызвать `sum<int>(4, 5)?`

- Что произойдёт, если вызвать `sum<int>(4, 5)`?

- Что произойдёт, если вызвать `sum<int>(4, 5)`? У типа `int` нет вложенного типа `iterator`, поэтому должна произойти ошибка компиляции

- Что произойдёт, если вызвать `sum<int>(4, 5)`? У типа `int` нет вложенного типа `iterator`, поэтому должна произойти ошибка компиляции
- Стандарт C++ говорит в таких случаях игнорировать те перегрузки, что вызывают *ошибку (failure)* в процессе *подстановки (substitution)* шаблонного параметра

- Что произойдёт, если вызвать `sum<int>(4, 5)`? У типа `int` нет вложенного типа `iterator`, поэтому должна произойти ошибка компиляции
- Стандарт C++ говорит в таких случаях игнорировать те перегрузки, что вызывают *ошибку (failure)* в процессе *подстановки (substitution)* шаблонного параметра
- Substitution Failure Is Not An Error, или **SFINAE**

- Что произойдёт, если вызвать `sum<int>(4, 5)`? У типа `int` нет вложенного типа `iterator`, поэтому должна произойти ошибка компиляции
- Стандарт C++ говорит в таких случаях игнорировать те перегрузки, что вызывают *ошибку (failure)* в процессе *подстановки (substitution)* шаблонного параметра
- **Substitution Failure Is Not An Error**, или **SFINAE**
- $\implies$ Выберется другая перегрузка `sum(T, T)`

- Можно узнать, есть ли у типа вложенный тип `iterator`:

```
template <typename T>
struct has_iterator {
    template <typename U>
    static char test(typename U::iterator*);

    template <typename U>
    static long test(...);

    static constexpr bool value =
        (sizeof(test<T>(nullptr)) == sizeof(char));
};

has_iterator<std::vector<int>>::value; // == true
```


- Можно проверить, если ли перегрузка некоторой функции от типа:

```
template <typename T>
struct is_printable {
    template <typename U>
    static char test(decltype(print(U))*);

    template <typename U>
    static long test(...);

    static constexpr bool value =
        (sizeof(test<T>(nullptr)) == sizeof(char));
};
```

- Можно использовать напрямую, без классов-обёрток:

```
template <typename T>
void show(const T& x, typename T::iterator* = nullptr) {
    std::cout << x;
}
```

- `std::enable_if` – шаблон, помогающий за счёт SFINAE специализировать функции по какому-либо условию

enable_if

- `std::enable_if` – шаблон, помогающий за счёт SFINAE специализировать функции по какому-либо условию

```
template <bool Condition, typename T>
struct enable_if {
    using type = T;
};

template <typename T>
struct enable_if<false, T> {};

template <typename T>
using enable_if_t = typename enable_if<T>::type;
```

```
template <typename T>
typename std::enable_if_t<!has_iterator<T>, void>
print(const T& x) {
    std::cout << x;
}

template <typename T>
typename std::enable_if_t<has_iterator<T>, void>
void print(const T& x) {
    for (const auto& v : x)
        std::cout << v;
}
```

Концепты (concepts)

- Появились в C++20

Концепты (concepts)

- Появились в C++20
- Изначально задумывались как сильное расширение системы типов C++ в сторону Haskell typeclasses и higher-order types

Концепты (concepts)

- Появились в C++20
- Изначально задумывались как сильное расширение системы типов C++ в сторону Haskell typeclasses и higher-order types
- В итоге свелись к аналогу **SFINAE/enable_if**, но с нормальным синтаксисом и с поддержкой со стороны компилятора

Концепты (concepts)

- Появились в C++20
- Изначально задумывались как сильное расширение системы типов C++ в сторону Haskell typeclasses и higher-order types
- В итоге свелись к аналогу **SFINAE/enable_if**, но с нормальным синтаксисом и с поддержкой со стороны компилятора
- **NB:** В первой версии концептов можно было задавать *аксиомы* на операции и типы (например, что сложение ассоциативно)

- Сам концепт – по сути, булевая константа, параметризованная типом, известная на момент компиляции

Концепты (concepts)

- Сам концепт – по сути, булевая константа, параметризованная типом, известная на момент компиляции

```
template <typename T>
concept Integral = std::is_integral_v<T>;

template <typename T>
concept Unsigned = Integral<T> && std::is_unsigned_v<T>;
```

Requires expression

- Для упрощения задания концептов есть особый синтаксис:
requires expression

Requires expression

- Для упрощения задания концептов есть особый синтаксис:
requires expression

```
template <typename T>
concept Comparable = requires (T a, T b) {
    a < b; // Проверяет, что есть подходящий оператор <
};
```

Requires expression

- Для упрощения задания концептов есть особый синтаксис: *requires expression*

```
template <typename T>
concept Comparable = requires (T a, T b) {
    a < b; // Проверяет, что есть подходящий оператор <
};
```

- Можно задавать несколько выражений, и проверять их типы:

```
template <typename T>
concept Comparable = requires (T a, T b) {
    { a < b } -> std::same_as<bool>;
    // или
    { a < b } -> std::convertible_to<bool>;
};
```

Requires expression

```
template <typename It>
concept ForwardIterator = requires (It it) {
    typename It::value_type;
    { *it } -> std::same_as<typename It::value_type>;
    { ++it } -> std::same_as<It>;
};
```

Использование концептов, requires clause

- Можно ограничить параметры шаблонной функции только теми, что удовлетворяют опеределённому концепту:

```
template <typename T>  
requires Comparable<T>  
T min(T x, T y) {  
    return (x < y) ? x : y;  
}
```


Использование концептов, requires clause

- Можно ограничить параметры шаблонной функции только теми, что удовлетворяют опеределённому концепту:

```
template <typename T>
requires Comparable<T>
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

- Можно комбинировать несколько концептов:

```
template <typename T>
requires Comparable<T> && Copyable<T>
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

Использование концептов, requires clause

- Есть упрощённый синтаксис для простого случая одного концепта:

```
template <Comparable T>
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

Использование концептов, requires clause

- Есть упрощённый синтаксис для простого случая одного концепта:

```
template <Comparable T>
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

- ...но его можно комбинировать и с другими концептами:

```
// То же самое, что и Copyable + Comparable
template <Comparable T>
requires Copyable<T>
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

Использование концептов, requires clause

- То же самое можно использовать для шаблонов классов:

```
template <typename T>  
requires Comparable<T>  
struct sorted_array {  
    // ...  
};
```

Использование концептов, requires clause

- То же самое можно использовать для шаблонов классов:

```
template <typename T>
requires Comparable<T>
struct sorted_array {
    // ...
};
```

```
template <Comparable T>
struct sorted_array {
    // ...
};
```

Requires requires

- Конструкция **requires**, позволяющая проверить набор условий на тип, называется *requires expression*:

```
requires (T a) { a < b; }
```

Requires requires

- Конструкция **requires**, позволяющая проверить набор условий на тип, называется *requires expression*:

```
requires (T a) { a < b; }
```

- Конструкция **requires**, проверяющая шаблонный параметр, называется *requires clause*:

```
...  
requires Comparable<T>  
...
```

Requires requires

- Конструкция **requires**, позволяющая проверить набор условий на тип, называется *requires expression*:

```
requires (T a) { a < b; }
```

- Конструкция **requires**, проверяющая шаблонный параметр, называется *requires clause*:

```
...  
requires Comparable<T>  
...
```

- Что, если мы хотим определить безымянный концепт и сразу его использовать?

- Что, если мы хотим определить безымянный концепт и сразу его использовать?

Requires requires

- Что, если мы хотим определить безымянный концепт и сразу его использовать? $\implies$ *requires requires*:

```
template <typename T>
requires requires (T x, T y) { x < y; }
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

Requires requires

- Что, если мы хотим определить безымянный концепт и сразу его использовать? $\implies$ *requires requires*:

```
template <typename T>
requires requires (T x, T y) { x < y; }
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

- Выражение `requires (T x, T y) { x < y; }` целиком – compile-time булево значение (requires expression)

Requires requires

- Что, если мы хотим определить безымянный концепт и сразу его использовать? $\implies$ *requires requires*:

```
template <typename T>
requires requires (T x, T y) { x < y; }
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

- Выражение `requires (T x, T y) { x < y; }` целиком – compile-time булево значение (requires expression)
- Выражение `requires Condition` ожидает compile-time булево значение в качестве параметра (requires clause)

- Чем концепты лучше, чем SFINAE + enable_if?

Концепты (concepts)

- Чем концепты лучше, чем SFINAE + enable_if?
- Быстрее компиляция

Концепты (concepts)

- Чем концепты лучше, чем SFINAE + enable_if?
- Быстрее компиляция
- Чище и понятнее код

Концепты (concepts)

- Чем концепты лучше, чем SFINAE + enable_if?
- Быстрее компиляция
- Чище и понятнее код
- Не полагаемся на хаки языка

- Зачем использовать концепты?

- Зачем использовать концепты?
- Специализация структур данных или алгоритмов по наборам типов

- Зачем использовать концепты?
- Специализация структур данных или алгоритмов по наборам типов
- Улучшение ошибок компиляции

Ошибки компиляции в шаблонах

- Один из недостатков шаблонов как вида полиморфизма – сложные ошибки компиляции

Ошибки компиляции в шаблонах

- Один из недостатков шаблонов как вида полиморфизма – сложные ошибки компиляции
- Пример: пытаемся отсортировать тип, у которого нет оператора сравнения

```
#include <algorithm>
#include <iterator>

struct point {
    int x;
    int y;
};

int main()
{
    point a[3] { {1,2}, {3,4}, {5,6} };
    std::sort(std::begin(a), std::end(a));
}
```

Ошибки компиляции в шаблонах

- Один из недостатков шаблонов как вида полиморфизма – сложные ошибки компиляции
- Пример: пытаемся отсортировать тип, у которого нет оператора сравнения

```
#include <algorithm>
#include <iterator>

struct point {
    int x;
    int y;
};

int main()
{
    point a[3] { {1,2}, {3,4}, {5,6} };
    std::sort(std::begin(a), std::end(a));
}
```

- Ожидание: ‘у класса `point` нет оператора `<`’

Ошибки компиляции в шаблонах

- Реальность:

[illegible]

- GCC 15: ошибка на 131 строку

Ошибки компиляции в шаблонах

- GCC 15: ошибка на 131 строку
- `std::sort` вызывает `std::__sort`, который вызывает `std::__introsort_loop`, который вызывает `std::__partial_sort`, который вызывает `std::__heap_select`, который вызывает `std::__make_heap`, который вызывает `std::__adjust_heap`, который вызывает `std::__push_heap`, который вызывает сравнение объектов типа `point`

Ошибки компиляции в шаблонах

- GCC 15: ошибка на 131 строку
- `std::sort` вызывает `std::__sort`, который вызывает `std::__introsort_loop`, который вызывает `std::__partial_sort`, который вызывает `std::__heap_select`, который вызывает `std::__make_heap`, который вызывает `std::__adjust_heap`, который вызывает `std::__push_heap`, который вызывает сравнение объектов типа `point`
- У компилятора нет информации о том, какой из приведённых вызовов на самом деле ошибочный, поэтому ему приходится выдавать весь стек вызова

Ошибки компиляции в шаблонах

- GCC 15: ошибка на 131 строку
- `std::sort` вызывает `std::__sort`, который вызывает `std::__introsort_loop`, который вызывает `std::__partial_sort`, который вызывает `std::__heap_select`, который вызывает `std::__make_heap`, который вызывает `std::__adjust_heap`, который вызывает `std::__push_heap`, который вызывает сравнение объектов типа `point`
- У компилятора нет информации о том, какой из приведённых вызовов на самом деле ошибочный, поэтому ему приходится выдавать весь стек вызова
- Компилятор не знает, какой ожидался оператор, поэтому ему приходится вывести список всех опробованных операторов, включая абсолютно нерелевантные (например, для итераторов по `std::vector`, или для строк)

Ошибки компиляции в концептах

- Добавим в `std::sort` НОВЫЙ КОНЦЕПТ `Sortable`:

```
template <typename It>
concept Sortable = requires (It x, It y) {
    { *x < *y } -> std::same_as<bool>;
};

template <Sortable It>
void sort(It begin, It end) {
    // ...
}
```

Ошибки компиляции в концептах

```
a.cpp: In function 'int main()':
a.cpp:27:18: error: no matching function for call to 'sort(point*, point*)'
   27 |         std::sort(std::begin(a), std::end(a));
      |         ~~~~~^~~~~
a.cpp:27:18: note: there is 1 candidate
a.cpp:17:6: note: candidate 1: 'template<class It> requires Sortable<It> void std::sort(It, It)'
   17 | void sort(It begin, It end)
      |      ^~~~~
a.cpp:17:6: note: template argument deduction/substitution failed:
a.cpp:17:6: note: constraints not satisfied
a.cpp: In substitution of 'template<class It> requires Sortable<It> void std::sort(It, It) [with It = point*]':
a.cpp:27:11:   required from here
   27 |         std::sort(std::begin(a), std::end(a));
      |         ~~~~~^~~~~
a.cpp:9:9:   required for the satisfaction of 'Sortable<It>' [with It = point*]
a.cpp:9:20:   in requirements with 'It x', 'It y' [with It = point*]
a.cpp:10:14: note: the required expression '(((* x) < (* y))' is invalid
   10 |         { *x < *y } -> std::same_as<bool>;
      |         ~~~~~^~~~~
cc1plus: note: set '-fconcepts-diagnostics-depth=' to at least 2 for more detail
```